

DODY DHARMA, PH.D

~ 90 MIN

COMPUTER NETWORKS • LECTURE 04

The Application Layer

DNS

The Web

Email



Today's agenda.

- | | | |
|----|---|---------|
| 01 | <h2>DNS</h2> <p>Names → addresses, the Internet's directory</p> | ~30 min |
| 02 | <h2>The Web</h2> <p>HTTP, caching, and the request/response model</p> | ~35 min |
| 03 | <h2>Email</h2> <p>SMTP, POP, IMAP — the oldest social network</p> | ~25 min |

01

DNS

Names → addresses, the Internet's directory

THE PROBLEM

Humans don't speak in numbers.

WHAT MACHINES NEED

167.205.1.34

A 32-bit IP address. Routable. Forgettable.

WHAT HUMANS USE

www.itb.ac.id

A name. Memorable. Stable across server moves.

Q. How do we map between the two?

DOMAIN NAME SYSTEM

DNS is the Internet's phone book.

A distributed database implemented in a hierarchy of many name servers, plus an application-layer protocol for resolving names.

SERVICE 01

Hostname → IP

The core lookup. `www.itb.ac.id`
→ `167.205.1.34`

SERVICE 02

Host aliasing

Canonical names, alias names,
CNAME records

SERVICE 03

Mail server

MX records point a domain at its
mail host

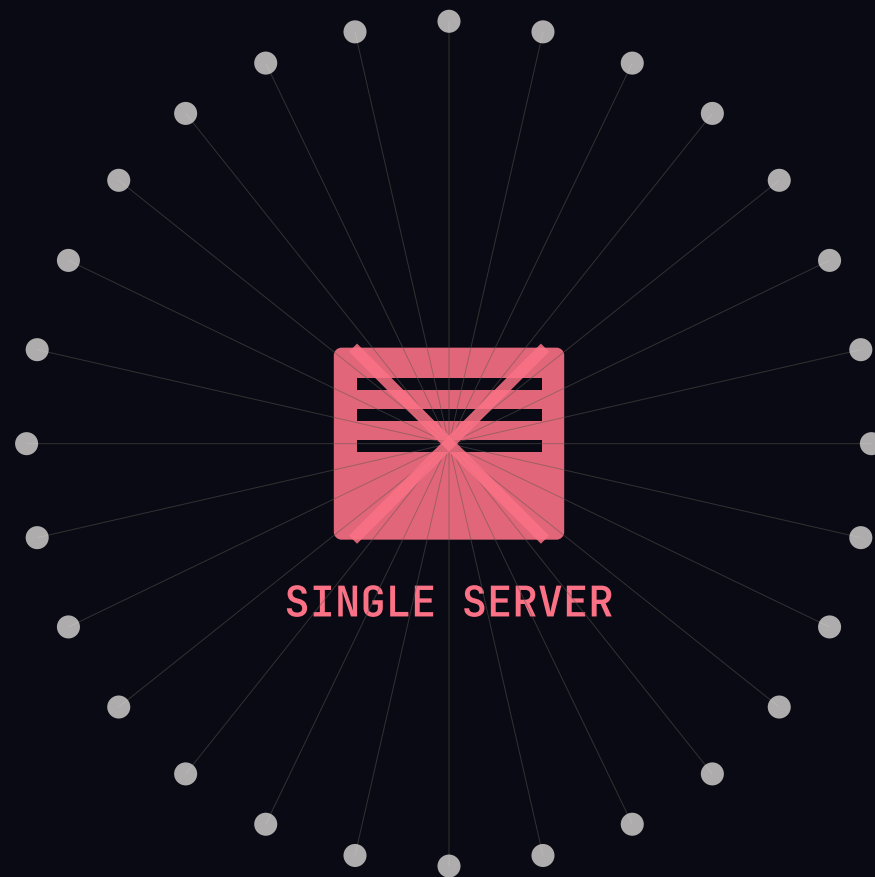
SERVICE 04

Load distribution

One name → many IPs, rotated
per query

A THOUGHT EXPERIMENT

Why not put everything on one server?



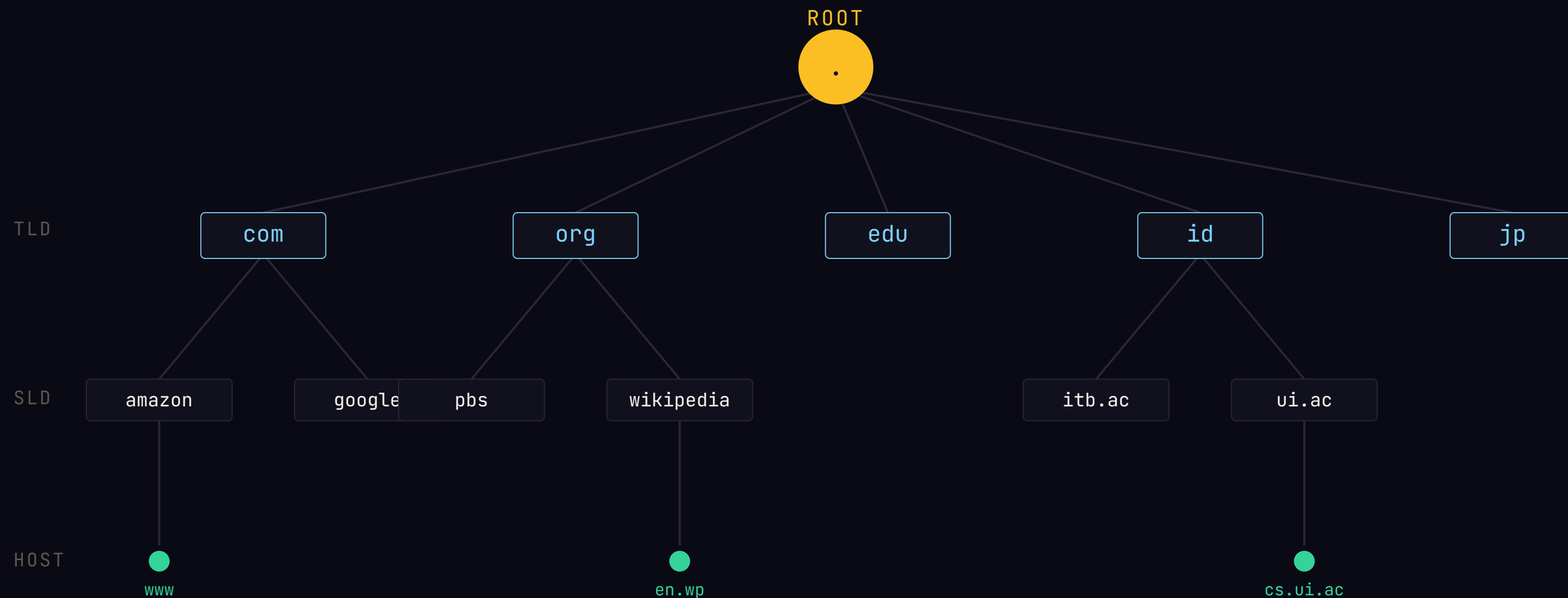
Four reasons it can't work:

- 01 **Single point of failure**
One outage takes the whole Internet offline.
- 02 **Traffic volume**
Billions of queries a second, on one box.
- 03 **Distance**
A user in Jakarta shouldn't query a server in Virginia.
- 04 **Maintenance**
Every change must be made on that one server.

→ **distribute it.**

THE NAMESPACE

An upside-down tree of names.



Read names right to left: **www.itb.ac.id** = host **www** in zone **itb.ac**, under TLD **id**, under root.

TOP-LEVEL DOMAINS

Roughly 250 TLDs sit just below the root.

Generic

Themed by purpose.

.com	.org	.net
.edu	.gov	.mil
.info	.biz	.io

Country-code

Two letters per ISO country.

.id	.jp	.uk
.de	.fr	.au
.sg	.ca	.nl

note Country TLDs often carry their own conventions — .ac.jp mirrors .edu, .co.uk mirrors .com. Names are case-insensitive.

THREE TIERS OF NAME SERVERS

Each tier knows just enough to delegate.

TIER 01



Root

Knows where to find every TLD server.

13 logically; hundreds of physical replicas worldwide via anycast.

TIER 02



TLD

Knows the authoritative servers for every domain under it.

Verisign runs .com; Educause runs .edu; PANDI runs .id.

TIER 03



Authoritative

Holds the actual records for an organization's hosts.

Run by the org or by a managed provider (e.g. Cloudflare, Route 53).

ONE MORE NODE

The local DNS server is your front door.



Not in the hierarchy.

Doesn't own a zone — it's a client of the hierarchy.

Every ISP, company, and university has one.

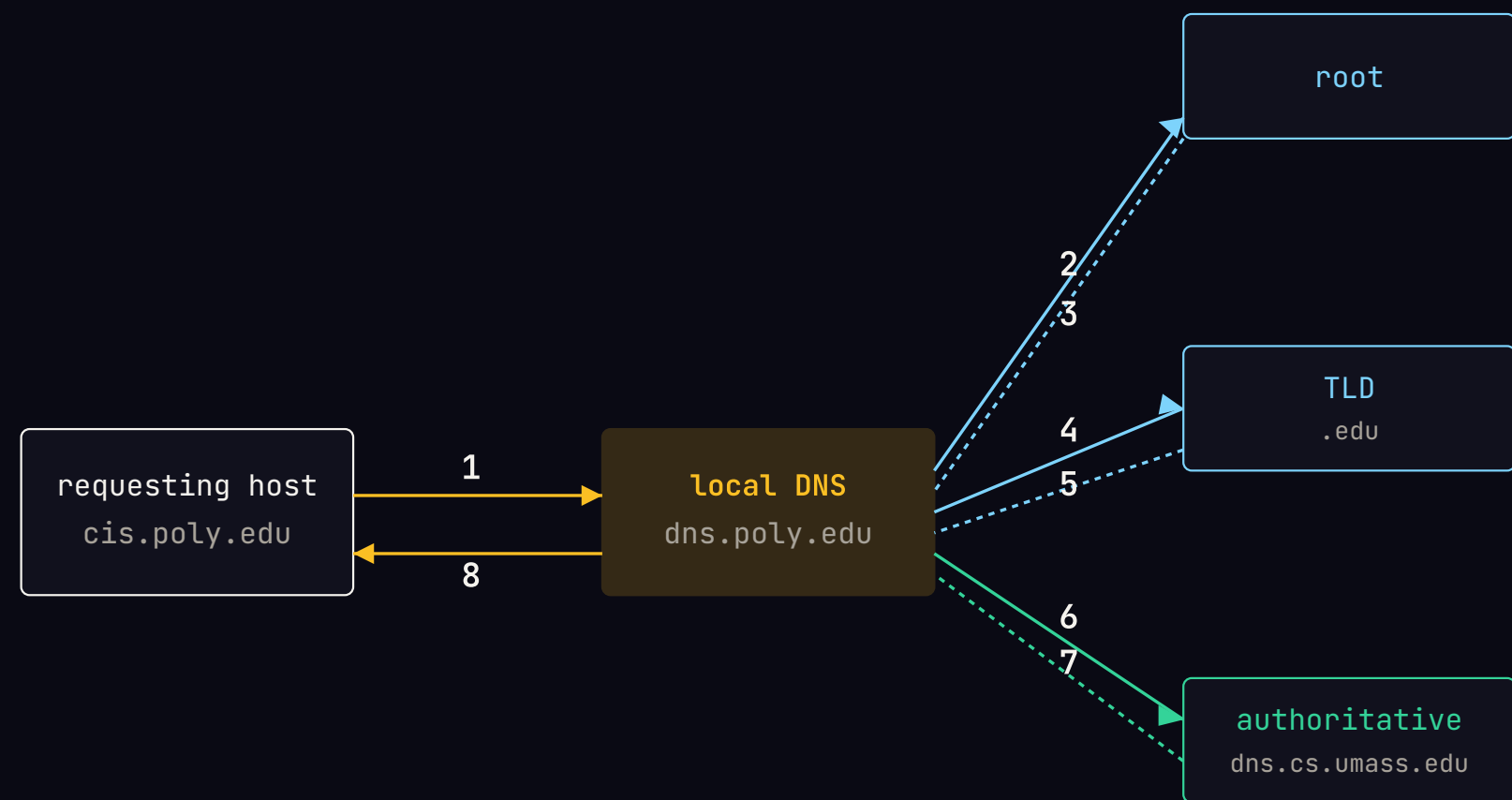
Often called the "default" name server.

Acts as a proxy.

Your host queries it; it queries the world; it caches the answer.

RESOLUTION MODE 01

Iterative — "I don't know, but ask them."

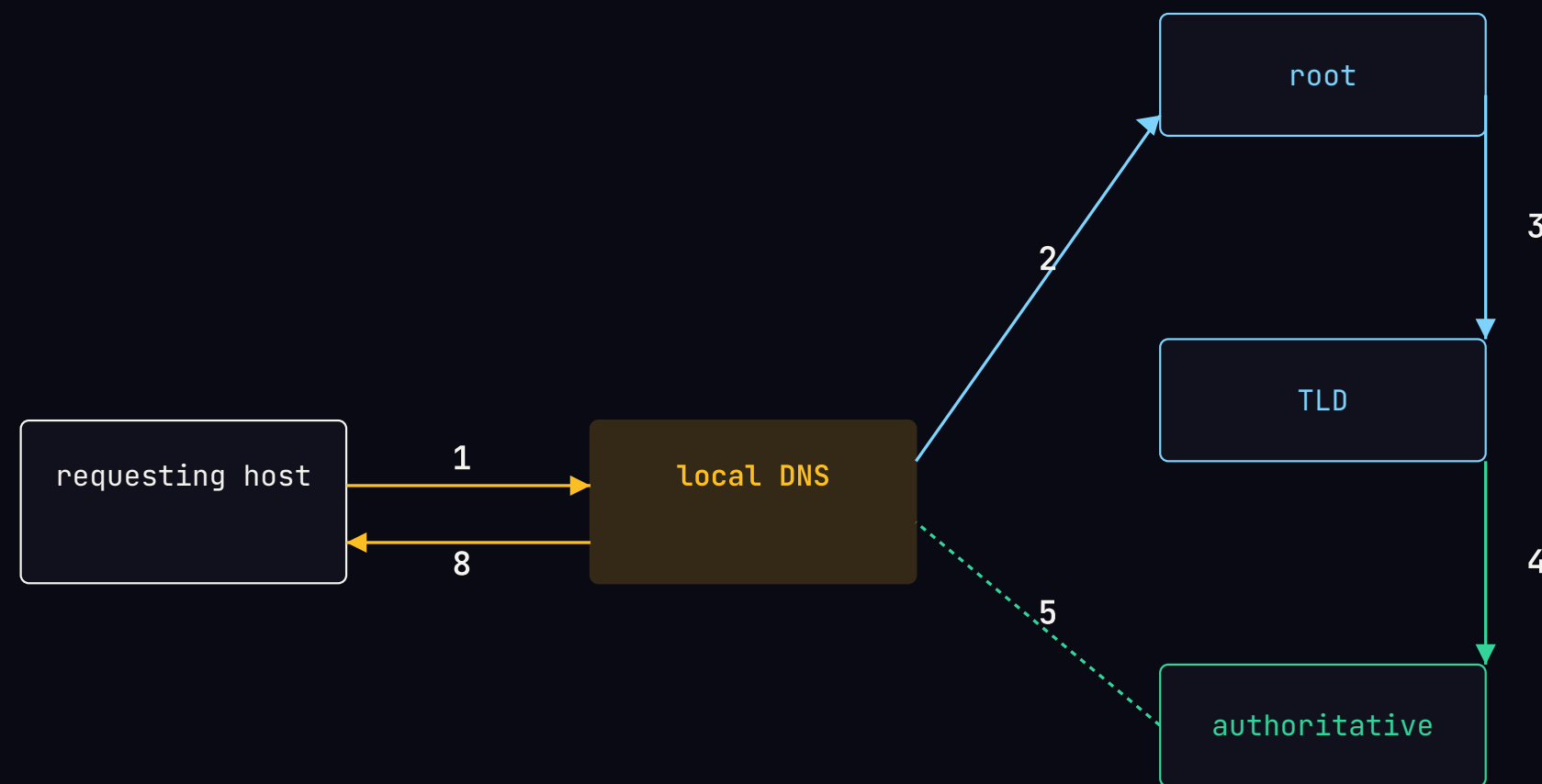


The local server walks the tree itself, getting referrals at each step.

- + **load is spread** across the hierarchy.
- **more round-trips** from the local server.

RESOLUTION MODE 02

Recursive — "I'll handle it; come back later."



Each contacted server resolves on the local server's behalf, passing the question deeper.

- + **fewer round-trips** for the local server.
- **heavy load** on upstream servers, so root and TLD typically refuse it.

PERFORMANCE

Every answer gets cached.

Why it works

- Once a name server learns a mapping, it stores the result.
- TLD records are usually cached at the local server, so root rarely sees traffic.
- Cache entries expire after the record's **TTL** (time-to-live).

TTL TRADE-OFF

long TTL Fewer queries · stale records on changes

short TTL Fresh records · more load on authoritative

Updates happen by waiting for TTL to expire — there is no global "push" today. (RFC 2136 designs a notify mechanism but it's rarely used end-to-end.)

WHAT'S ACTUALLY STORED

The DNS database is a list of resource records.

Every entry follows the same five-field shape:

NAME	TTL	CLASS	TYPE	VALUE
The domain name the record describes.	How long it can be cached, in seconds.	Almost always IN for Internet.	What kind of information this is.	The data – meaning depends on type.

EXAMPLE

www.itb.ac.id 3600 IN A 167.205.1.34

COMMON TYPES

Eight record types do most of the work.

A	Hostname → IPv4 address	www.itb.ac.id IN A 167.205.1.34
AAAA	Hostname → IPv6 address	www.itb.ac.id IN AAAA 2404:8000::34
NS	Domain → authoritative name server	itb.ac.id IN NS ns1.itb.ac.id
CNAME	Alias → canonical name	www.kaskus.com IN CNAME www.kaskus.us
MX	Domain → mail server	yahoo.com IN MX smtp.yahoo.com
PTR	IP → hostname (reverse)	34.1.205.167.in-addr.arpa IN PTR www.itb.ac.id
TXT	Free-form text (SPF, verification, ...)	itb.ac.id IN TXT "v=spf1 ..."

THE WIRE PROTOCOL

Queries and replies share one format.

Header	id (16-bit) · flags · counts of each section below	Identification matches reply to query. Flags say: query/reply, recursion-desired, recursion-available, authoritative.
Question	name + type – what's being asked	Repeated for each question in this packet.
Answer	RRs that answer the question	
Authority	RRs pointing to authoritative name servers	
Additional	Extra helpful records (e.g. glue A records)	

END - TO - END

How a new domain enters DNS.

Scenario: a new startup, **networkutopia.com**.

1 Register the name with a registrar (e.g. Network Solutions, Namecheap).

You provide the names and IPs of your two authoritative name servers.

2 Registrar inserts two RRs into the .com TLD server.

(networkutopia.com , dns1.networkutopia.com , NS) · (dns1.networkutopia.com , 212.212.212.1 , A)

3 You create the actual records on your authoritative server.

An A record for www.networkutopia.com · an MX record for networkutopia.com

4 TTLs expire across the world's caches.

Within hours, queries for your name resolve everywhere.

DEMO

Watching a name turn into an address.

```
● ● ● ~ — bash
```

```
$ ping www.itb.ac.id
```

```
PING www.itb.ac.id (167.205.1.34): 56 data bytes
```

```
64 bytes from 167.205.1.34: icmp_seq=0 ttl=57 time=14.6 ms
```

```
64 bytes from 167.205.1.34: icmp_seq=1 ttl=57 time=13.7 ms
```

```
64 bytes from 167.205.1.34: icmp_seq=2 ttl=57 time=26.5 ms
```

```
$ dig +short www.itb.ac.id A
```

```
167.205.1.34
```

```
$ dig +trace www.itb.ac.id
```

```
.          IN  NS  a.root-servers.net.
```

```
id.        IN  NS  ns1.id.
```

```
ac.id.     IN  NS  ns1.ac.id.
```

```
itb.ac.id IN  NS  ns1.itb.ac.id.
```

```
www.itb.ac.id IN  A  167.205.1.34
```

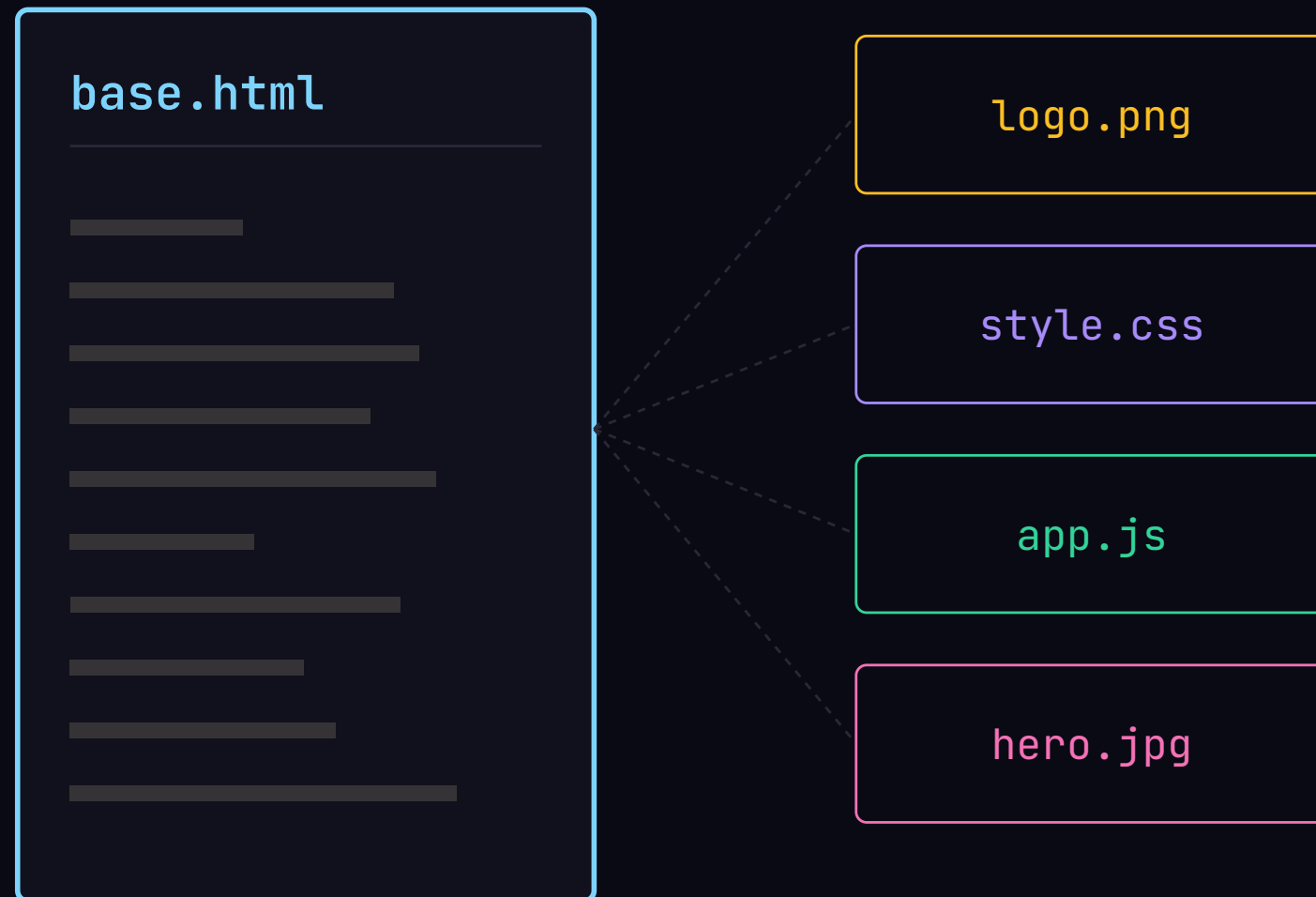
02

Web

HTTP, requests and responses, the rest of the web

VOCABULARY

A page is one HTML file plus its objects.



Each object has a URL.

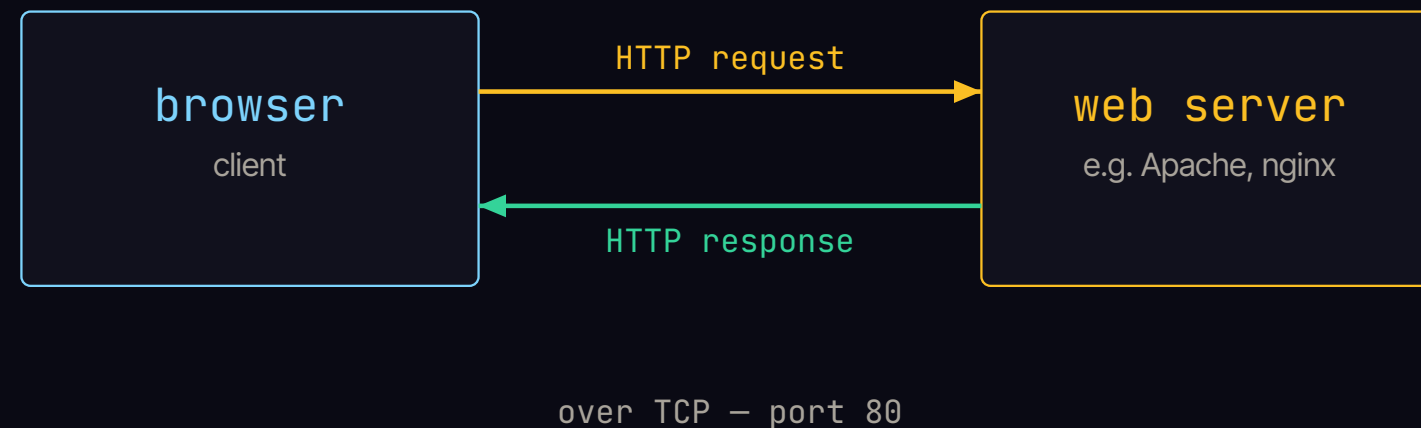
`www.someschool.edu/someDept/pic.gif`

• host name • path

Objects can be HTML, images, scripts, fonts, audio — anything addressable.

THE WEB'S PROTOCOL

HTTP is a request/response conversation.



Hypertext Transfer Protocol

The Web's application-layer protocol — RFCs 1945, 2616, 7230.

Client/server model

The browser asks; the web server answers.

Runs over TCP, port 80

Client opens a socket; server accepts; messages flow; connection closes.

Stateless

The server keeps no memory of past requests. State lives elsewhere (cookies).

A DESIGN CHOICE

Stateless is a feature, not a limitation.

WHAT IT MEANS

Each HTTP request stands alone. The server has no memory of what you asked five seconds ago.

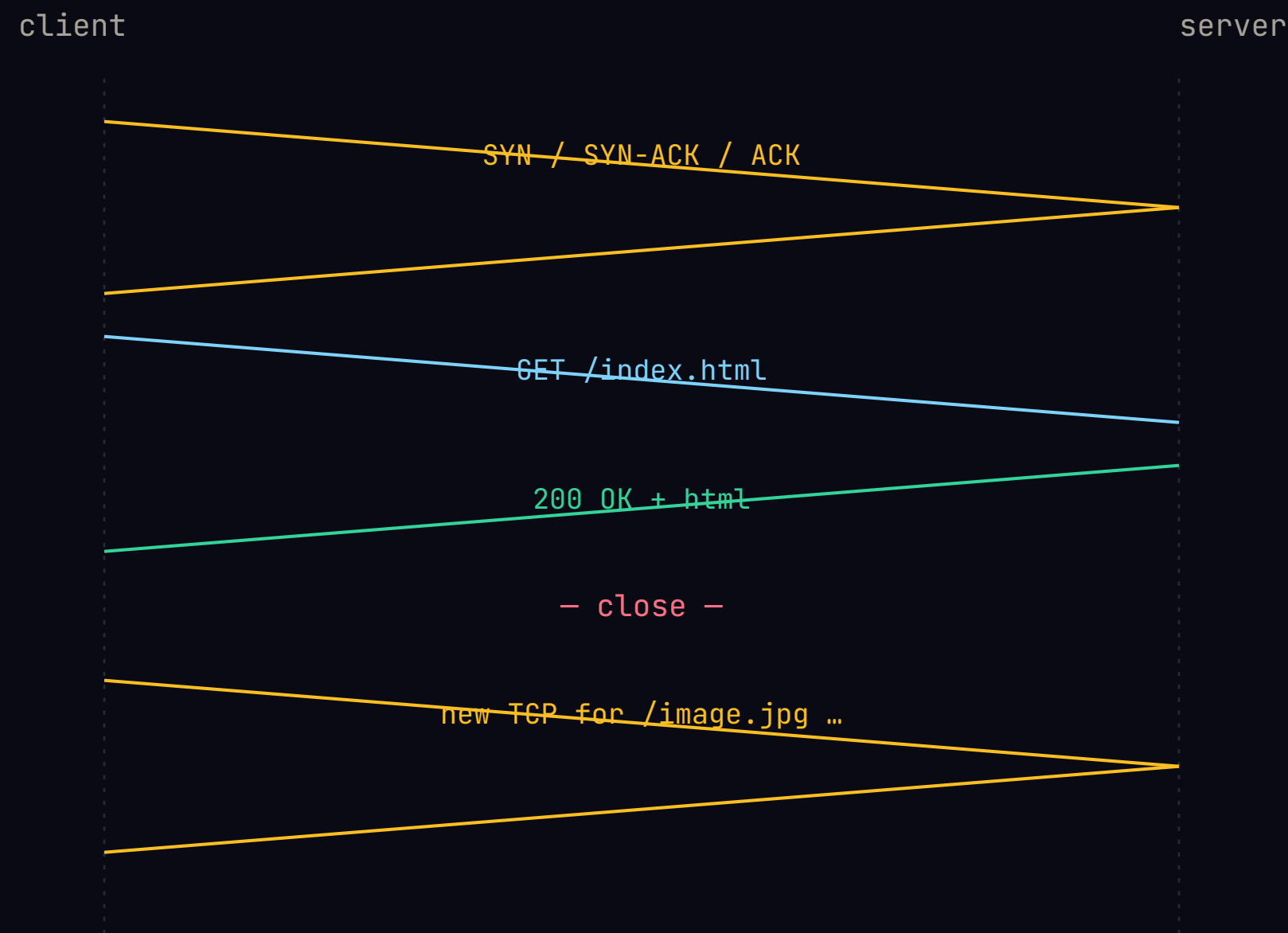
WHY IT'S GOOD

Servers stay simple. They scale horizontally. A crash recovers cleanly — no state to reconcile between client and server.

aside Protocols that *do* maintain state are far more complex: history must be persisted, and after a crash, both sides may disagree on what the state should be.

HTTP/1.0 DEFAULT

Non-persistent — one object per connection.



For each object, the client and server perform a fresh TCP handshake, exchange one request/response pair, then close.

A page with 10 images means **11 connections**. That cost adds up.

THE MATH

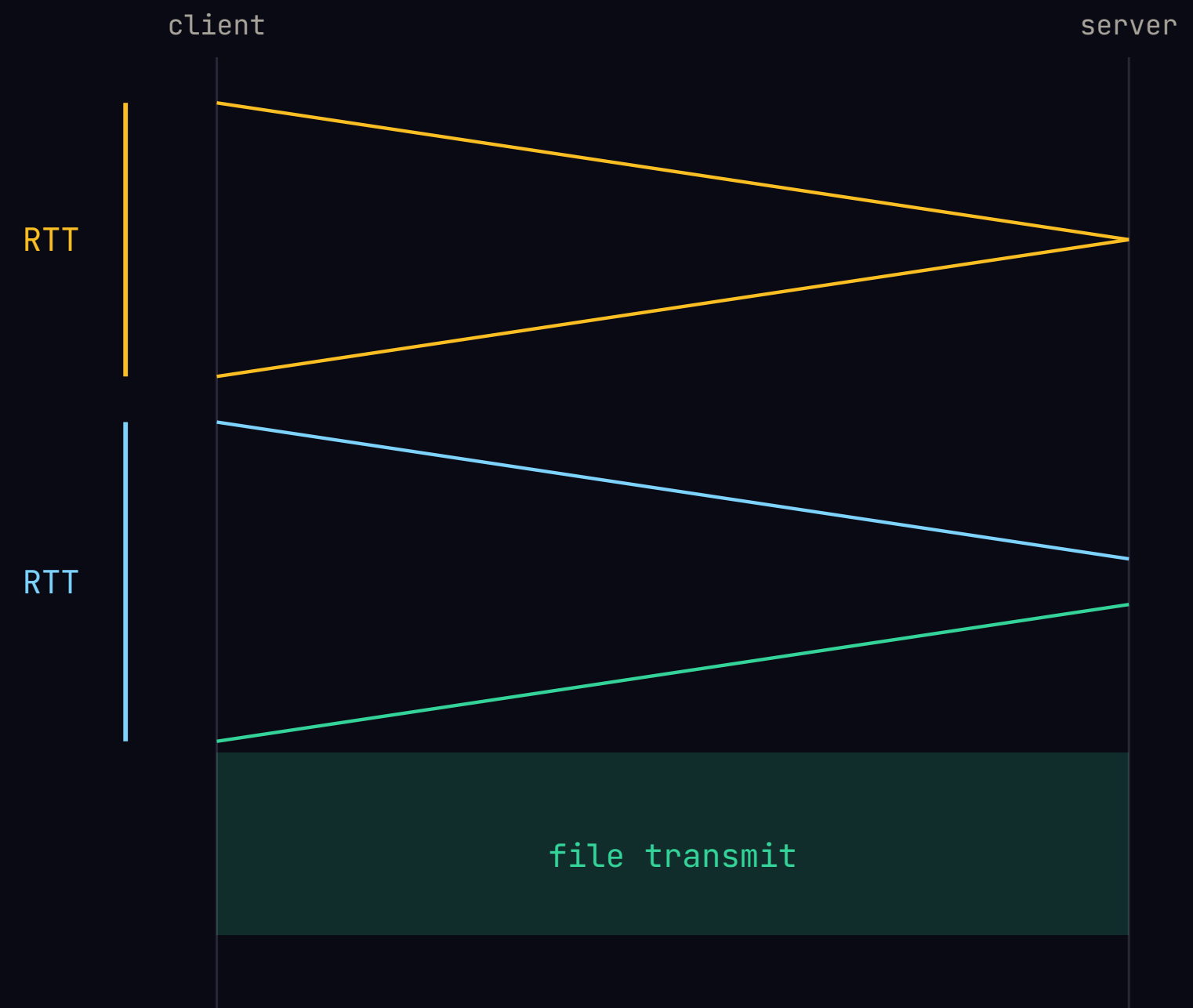
Each object costs 2 RTT plus transmit time.

RTT round-trip time

small packet, client → server → client

$$\text{total} = 2 \cdot \text{RTT} + \text{transmit}$$

One RTT to set up TCP. One RTT to send the request and receive the first byte back. Then the file streams through.



HTTP/1.1 DEFAULT

Persistent — keep the connection open.

NON-PERSISTENT COSTS

- 2 RTT per object
- OS overhead per TCP connection
- Browsers compensate by opening many parallel connections

PERSISTENT WINS

- Server keeps the socket open after the response
- Client pipelines next request as soon as it parses a reference
- As little as **1 RTT** for all referenced objects

Modern browsers default to persistent + multiplexing. HTTP/2 and HTTP/3 take this further still.

THE WIRE FORMAT

An HTTP request is plain ASCII.

```
GET /somedir/page.html HTTP/1.1
```

```
Host: www.someschool.edu
```

```
User-agent: Mozilla/5.0
```

```
Connection: keep-alive
```

```
Accept-Language: en
```



request line

method · URL · protocol version

header lines

name: value pairs — host, agent, encoding, language...

blank line

Carriage-return line-feed signals end of headers.

entity body (optional)

Used by POST, PUT for form data or uploads.

METHOD TYPES

HTTP defines eight verbs.

GET HTTP/1.0 Retrieve a resource. Idempotent.

POST HTTP/1.0 Submit data — form input goes in the body.

HEAD HTTP/1.0 Like GET, but server omits the body.

PUT HTTP/1.1 Upload a file to the URL specified.

DELETE HTTP/1.1 Remove the resource at the URL.

OPTIONS HTTP/1.1 Ask which methods the server supports.

TRACE HTTP/1.1 Echo the request back — for debugging.

CONNECT HTTP/1.1 Establish a tunnel to the resource (HTTP/2)

THE REPLY

Status line, headers, body.

```
HTTP/1.1 200 OK
Date: Tue, 28 Apr 2026 12:00 GMT
Server: Apache/2.4 (Unix)
Last-Modified: Mon, 27 Apr...
Content-Length: 6821
Content-Type: text/html
⏏
<html>...</html>
```

status line

protocol · status code · status phrase

header lines

metadata about the body — type, length, freshness

body

the actual object — HTML, JSON, image bytes...

STATUS CODES

The first digit tells you the family.

1xx

Informational

2xx

Success

3xx

Redirect

4xx

Client error

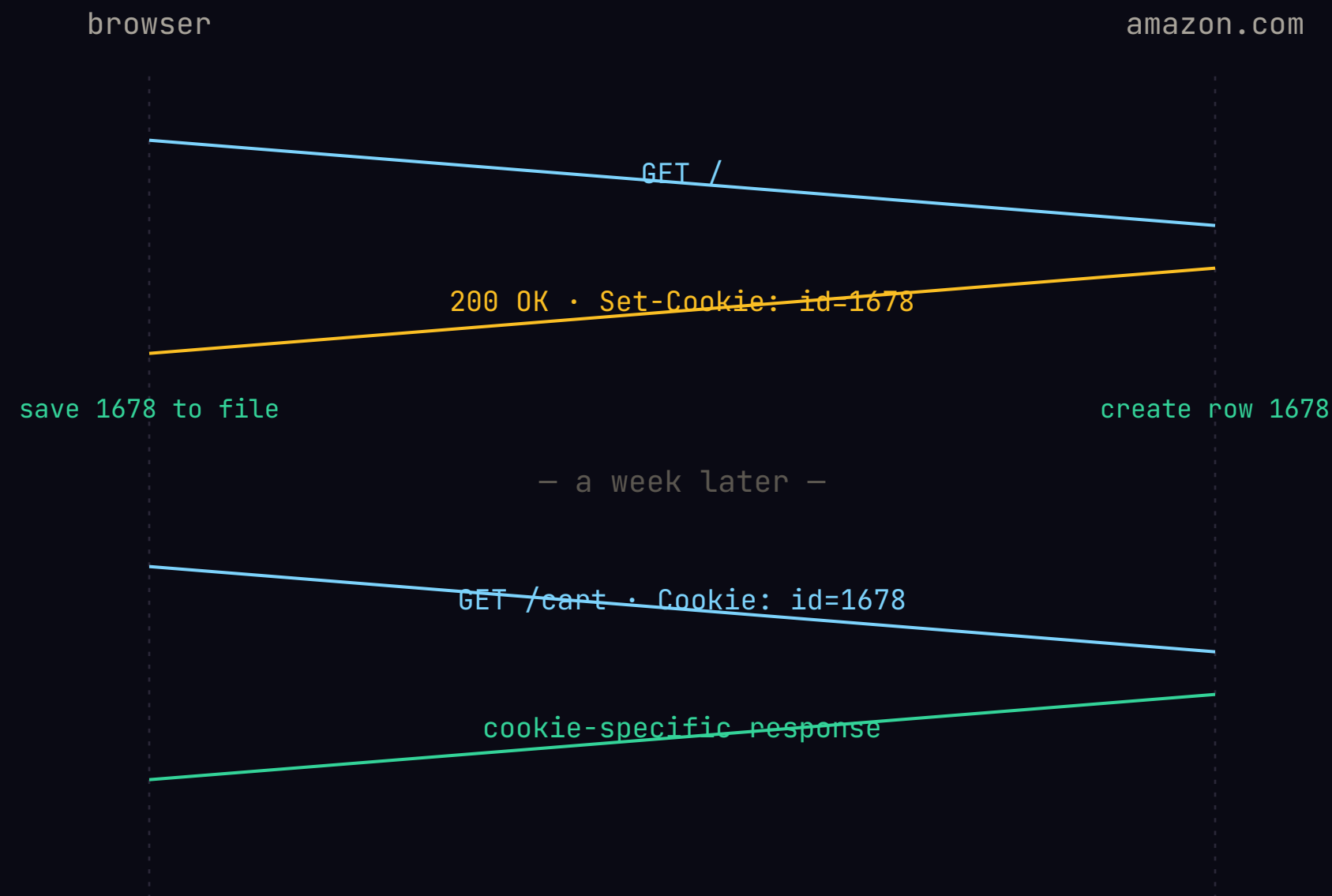
5xx

Server error

200	OK	Request succeeded; object follows.
301	Moved Permanently	Use the new Location: from now on.
304	Not Modified	Cache is fresh — keep using it.
400	Bad Request	Server couldn't parse the message.
404	Not Found	No such object on this server.
500	Internal Server Error	Something blew up handling the request.
505	HTTP Version Not Supported	Server doesn't speak that HTTP version.

ADDING STATE ON TOP

Cookies make a stateless protocol feel stateful.



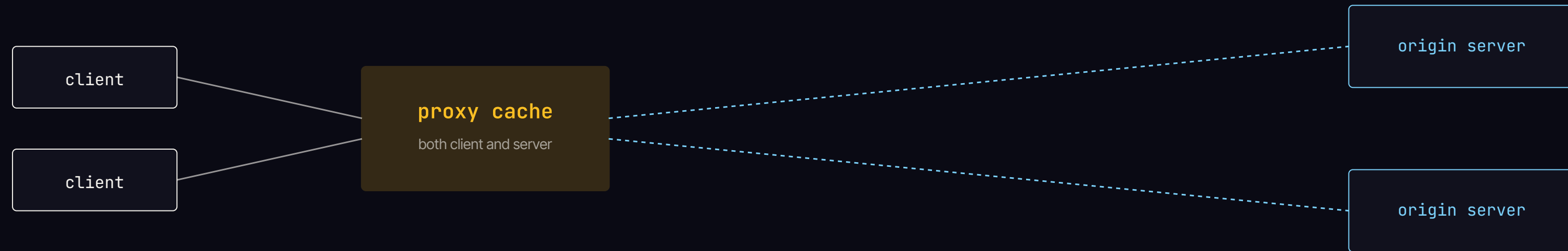
Four moving parts:

- Set-Cookie** in the response header
- Cookie header** in subsequent requests
- Cookie file** kept by the browser
- Backend database** keyed by the ID

Useful for: authorization, shopping carts, recommendations, session state. The privacy implications are real — sites learn a lot about you.

PROXIES & CACHES

A web cache sits between you and the origin.



cache hit → return locally · cache miss → fetch from origin, store, return

Faster

Cache hits return in milliseconds.

Cheaper

Reduce traffic on the access link.

Universal

Levels the playing field for small content providers.

A WORKED EXAMPLE

Cache or upgrade — what's cheaper?

BASELINE LAN util 15% Access link 100% Total delay minutes Access link saturated → multi-second queueing.	UPGRADE LINK Access link 1.5 → 10 Mbps LAN util 15% Access util 15% Works, but the upgrade is expensive.	INSTALL CACHE Hit rate 0.4 Access util 60% (no queueing) Avg delay < 1.4 s Same effect, fraction of the cost.
--	---	---

Setup: 100 kB avg object · 15 req/s from clients · 2 s round-trip to origin

CACHE FRESHNESS

Don't send the object if I already have it.

CACHE → SERVER

```
GET /index.html HTTP/1.1
```

```
If-Modified-Since:
```

```
Mon, 21 Apr 2026 12:00 GMT
```

STILL FRESH

```
HTTP/1.1 304 Not Modified
```

```
(no body)
```

CHANGED

```
HTTP/1.1 200 OK
```

```
<new data...>
```

The cache asks "has it changed since this date?" The server either returns a tiny **304** or the full object. Same idea: ETag + If-None-Match.

DEMO

You can speak HTTP yourself.

```
●●● ~ — telnet  
  
$ telnet cis.poly.edu 80  
Trying 128.238.32.27...  
Connected to cis.poly.edu.
```

```
GET /~ross/ HTTP/1.1  
Host: cis.poly.edu
```

```
HTTP/1.1 200 OK  
Server: Apache  
Content-Type: text/html  
Content-Length: 4218
```

```
<!DOCTYPE html>  
<html>...</html>
```

telnet opens a TCP socket on port 80. Type the request — the server responds in plain text.

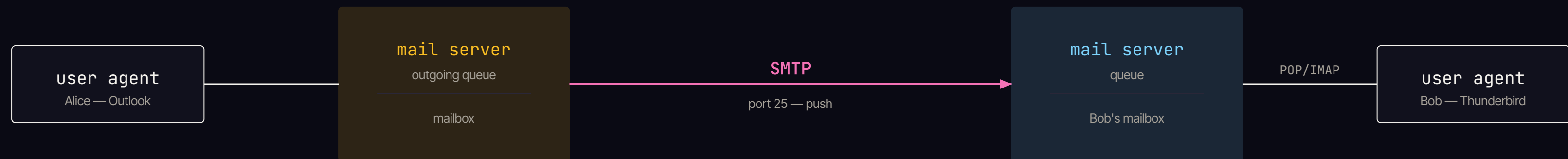
03

Email

SMTP, POP, IMAP — and why email feels old

THE SYSTEM

Email has three moving parts.



User agents

Compose, read, manage. Outlook, Thunderbird, Gmail web.

Mail servers

Hold mailboxes. Run SMTP for sending; POP/IMAP for retrieval.

SMTP

The push protocol that moves messages between servers.

SIMPLE MAIL TRANSFER PROTOCOL

SMTP is push, ASCII, and old.

RFC 2821 (1982 → today)

Older than the Web.

Runs over TCP, port 25

Reliable transfer between mail servers.

Direct: sender server → receiver server

No intermediate relays in the modern setup.

7-bit ASCII only

Binary attachments need MIME encoding.

Three phases

Handshake · message transfer · closure.

COMMAND SHAPE

HELO crepes.fr

MAIL FROM: <alice@crepes.fr>

RCPT TO: <bob@hamburger.edu>

DATA

QUIT

Server replies with three-digit status codes — same idea as HTTP.

WALKTHROUGH

Alice sends a message to Bob.

- 1 Alice composes in her UA and addresses bob@someschool.edu.
- 2 UA hands the message to her mail server; it lands in the outgoing queue.
- 3 Her server (SMTP client) opens TCP to Bob's mail server (port 25).
- 4 SMTP client sends the message over the connection.
- 5 Bob's mail server places the message in Bob's mailbox.
- 6 Bob's UA pulls it down via POP or IMAP and shows it.

LIVE TRANSCRIPT

An SMTP conversation, end to end.

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

SMTP VS HTTP

Two ASCII protocols, opposite philosophies.

	HTTP	SMTP
Direction	Pull	Push
Initiator	Client wants something	Sender pushes message
Connections	Persistent (1.1)	Persistent
Encoding	Any bytes	7-bit ASCII
Objects	Each in its own response	All bundled in multipart message

THE MESSAGE ITSELF

Header lines, blank line, body.

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Lunch?
MIME-Version: 1.0
Content-Type: text/plain
↵
Hi Bob — sandwiches at 1?
— Alice
```

RFC 822 — text format

Header lines (To, From, Subject...), blank line, body. Distinct from SMTP commands.

MIME — Multipurpose extensions

Lets a 7-bit channel carry images, audio, video, attachments. Adds Content-Type and Content-Transfer-Encoding.

Multipart messages

Body split by a boundary marker — text + HTML + attachment in one envelope.

READING MAIL

SMTP delivers; another protocol fetches.

SMTP only moves mail *between servers*. To pull from your server to your laptop, you need an access protocol.

POP3

RFC 1939

Authorize, then download.

Mode	Download (and delete or keep)
State	Stateless across sessions
Folders	No – local on client
Best for	One device only

IMAP

RFC 1730

Keep messages on the server.

Mode	Synchronize, don't move
State	Stateful – folders, flags
Folders	Yes – on the server
Best for	Multiple devices

HTTPS

Webmail

The browser is the client.

Mode	Web app over HTTP
State	Cookies + server-side
Folders	Yes – server-side
Best for	Gmail, Outlook.com, Yahoo

POP3 SESSION

Authorize · transact · quit.

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
C: dele 1
C: quit
S: +OK POP3 server signing off
```

DEMO

Sending a mail with no client at all.

```
$ openssl s_client -connect smtp.gmail.com:587 -starttls smtp
S: 250 smtp.gmail.com at your service
C: AUTH LOGIN
S: 334 VXNlcm5hbWU6
C: <base64 username>
S: 334 UGFzc3dvcmQ6
C: <base64 password>
S: 235 Accepted
C: MAIL FROM:<alice@gmail.com>
C: RCPT TO:<bob@example.com>
C: DATA
S: 354 Go ahead
C: Subject: hello from openssl
C: .
S: 250 OK
C: QUIT
```

Modern providers wrap SMTP in TLS (port 465 / 587 with STARTTLS) and require AUTH — but the verbs are the same.

ONE SLIDE, THREE PROTOCOLS

Where they sit, what they push, what they pull.

	PURPOSE	TRANSPORT	DIRECTION	STATEFUL?
DNS	Names → IPs	UDP/53 (TCP fallback)	Pull (query/reply)	No — caches by TTL
HTTP	Web objects	TCP/80 (TLS:443)	Pull (request/response)	No — cookies add state
SMTP	Email transfer	TCP/25 (TLS:465/587)	Push (sender → receiver)	No — multi-phase session
POP/ IMAP	Mailbox access	TCP/110, 143 (+TLS)	Pull	POP no · IMAP yes

Questions?

Try the demos yourself — `dig`, `telnet`, `openssl s_client`. The protocols you saw today are the same ones every machine on the Internet still speaks.